



ADR-005: taking a file from somebody, without becoming a place files are kept

Status: increment 1 **built and measured** 2026-09-08. Increment 2 is a proposal. **Constrained by** [ADR-001](#).

Context

Goal 2 of this project is *managing the library without launching YARG*. The browse page shipped 2026-09-07 and closed the reading half of that: search, sort, per-song metadata, the scanner's issues, a download link. What it cannot do is answer the question people actually arrive with — **"why won't this song work?"** — for a file that is not in the library yet.

Today the only way to get that answer is to copy the file onto the server's disk, restart or rescan, and read the log. That is a shell session, on the machine, for a question the scanner can answer in milliseconds.

The roadmap has called this *"Ingest from the browser — drop an archive in, watch it scan, see the verdict"* since Phase 4 was written, and [docs/API.md](#) has said ingest is "not here yet" rather than "not going to happen". So the direction is not in question. What needed deciding is the posture, because this server is **unauthenticated by design and read-only by design**, and accepting an upload touches both of those words.

Decision

Split the feature at the point where it stops being reversible, and build only the first half.

Increment 1 — check, and keep nothing. BUILT.

`POST /api/v1/check?name=<filename>` takes an archive as the request body, scans it, and answers with the verdict. It writes nothing to the library, adds nothing to the index, and deletes the staged upload before the response is written.

That is the whole feature, and it is worth being precise about how small it is:

Reads the body	yes, bounded by <code>check_max_bytes</code> (default 128 MiB)
Writes to <code>--songs</code>	never ; the library is untouched and on the live deployment is mounted <code>ro</code>
Writes to <code>--data</code>	one temp file under <code>data/check/</code> , deleted in a <code>defer</code>
Changes the index	no
Survives the request	nothing

It is OFF by default, and that is the decision that took the most thought. `browse_ui` defaults ON because it shows what `/api/v1/songs` already served to anyone who could reach the port — a new *view* of existing surface. This is different in kind: it accepts a large upload and spends CPU and temporary disk on whoever asks. **An existing deployment must not acquire that by being upgraded.** One line of config turns it on, and the server says so at start.

When it is off the route is **not registered at all**, so a disabled server answers 404 rather than 403. A 403 tells the caller the feature exists.

The verdict must be the SAME verdict

The handler does not implement its own idea of what a song is. It calls `scan.ScanFile`, which was extracted from `WalkLibrary` in the same change and is now the one place the file-shape dispatch lives — console package by suffix, container by extension, `.sng` otherwise.

This is the point of the increment. A second implementation of "what is this file" would agree with the first for a while and then quietly stop, and the disagreement would show up as *"the checker said it was fine and the library dropped it"* — the exact failure this project has already paid for twice, in the packcache path and in the two sync clients. `TestCheckAgreesWithALibraryScan` uploads a real `.sng`, then builds a library from the same bytes and compares the chart hashes.

The uploader's filename is a lookup key, not a filename

`?name=` exists because the reader is chosen by extension, exactly as a library walk chooses it from the name on disk. **Only the extension is used.** The name never reaches the filesystem: the body goes to a temp file whose name `os.CreateTemp` picks.

This is the sync-client lesson pointed inward. Those clients trusted a server to name files on the player's disk and that was a remote arbitrary file write; this is the same shape with the arrow reversed, and it would have been a natural thing to get wrong — joining the uploaded name onto a staging directory looks harmless.

The test for it had to be written twice, which is worth recording. The first version listed the staging directory's parent before and after and compared. It **passed against an implementation that really did join the name onto the staging path**, because that implementation created the

file outside, and then the handler's own cleanup deleted it again before the test looked. Identical listings, green test, a server that wrote exactly where it was told.

What actually gets damaged is a file that was *already there*, because `os.Create` truncates. So the test now writes a sentinel where a traversal would land and asserts it still says what it said. Both plausible buggy implementations — joining onto the staging path, and using the name directly — now fail it, and the failure message shows the operator's file being deleted outright rather than merely overwritten.

Increment 2 — keep the file. PROPOSED, not built.

Storing an accepted upload is a different decision and is deliberately not made here. What it would have to answer:

- **Where.** `--songs` is read-only in normal operation and mounted `ro` on the live deployment, so "write it into the library" is not a small change to a flag; it is a change to how the server is deployed. A separate `--incoming` directory that the index also walks is the likelier shape.
- **Who.** An unauthenticated write endpoint on a LAN is a different risk from an unauthenticated read one. Authentication is the other thing `API.md` lists as not here yet, and this is the first feature that actually needs it.
- **What it means for identity.** Two uploads of the same chart in different packages are already a thing the server has an answer for (300 Multiple Choices). Ingest has to not quietly break it.
- **Rescan.** The index is built at start and held in memory (ADR-002). An accepted upload that does not appear until a restart is a worse experience than no ingest at all.

None of those are hard. They are just decisions, and increment 1 is useful without any of them: *"tell me whether this file will work"* is the question people actually have, and it is answerable with no state change at all.

Consequences

- The server can now be asked a question it could not answer before, and answering it costs nothing permanent.
- The dispatch that decides what a file IS has exactly one implementation, used by both the library walk and the endpoint. That is a small refactor with an outsized payoff.
- `--data` now has a `check/` subdirectory, created at start when the feature is on, and swept at start for anything a previous run's crash left behind. "Keeps nothing" is a promise, and a promise that only holds when the process exits cleanly is not the promise that was made.
- The live vault2 deployment is **unchanged**: the feature is off, and turning it on is a config line plus a restart.
- Nothing here moves toward "control a running game", which remains out of scope.

What this deliberately does not do

- **No multipart.** A raw body plus `?name=` is what a browser's `fetch` sends for a dropped `File` with no ceremony, and what `curl --data-binary` sends. Multipart would add a parser between an untrusted client and this server for no gain.
- **No authentication.** Adding a token to one endpoint of an otherwise unauthenticated server would be security theatre; the endpoint is off by default instead, and authentication remains a whole-server decision.
- **No storage.** See increment 2.
- **No console-package handling.** A `_rb3con` is refused from the NAME, before a byte of the body is read, with the same reason a library scan gives. Decrypting one is a permanent non-goal and always will be.